

Fundamentos de TypeScript

José Vicente Berná Martínez

Profesor Titular de Universidad

Dpto. Tecnología Informática y Computación



Universitat d'Alacant
Universidad de Alicante



Grau en Enginyeria Multimèdia
Grado en Ingeniería Multimedia

Índice de contenidos

1.	Preambulo	4
2.	Tipos básicos y conceptos generales	4
2.1.	¿Qué es TypeScript y por qué lo utilizamos?	4
2.2.	Declaración de variables: tipado explícito vs. tipado inferido	4
2.2.1.	Tipado explícito (anotación de tipo)	4
2.2.2.	Tipado inferido	4
2.3.	Los tipos primitivos en detalle	5
2.3.1.	string (cadenas de texto)	5
2.3.2.	number (números)	5
2.3.3.	boolean (booleanos)	5
2.4.	El tipo any: ¿qué es y por qué se debe evitar?	5
2.4.1.	¿Por qué existe?	5
2.4.2.	¿Por qué se considera una mala práctica en código nuevo?	6
2.4.3.	La alternativa segura: unknown	6
3.	Objetos, arreglos e interfaces	6
3.1.	Arreglos (arrays) en profundidad	6
3.1.1.	Sintaxis básica de un arreglo	7
3.1.2.	Arreglos con tipos mixtos (unión de tipos)	7
3.1.3.	Arreglos inmutables (readonly)	7
3.2.	Objetos literales e inferencia de estructura	7
3.2.1.	¿Por qué TypeScript bloquea la adición de propiedades?	8
3.3.	Interfaces	8
3.3.1.	Declaración y uso básico	8
3.3.2.	Propiedades opcionales (?)	8
3.3.3.	Propiedades de solo lectura (readonly en interfaces)	9
4.	Funciones y sus argumentos	9
4.1.	Funciones básicas: tipado de entradas y salidas	9
4.1.1.	¿Por qué es importante tipar el retorno?	10
4.1.2.	El tipo de retorno void	10
4.2.	Funciones de flecha (arrow functions)	10
4.3.	Argumentos opcionales y por defecto	11
4.3.1.	Argumentos opcionales (?)	11

4.3.2.	Argumentos por defecto (valores predeterminados)	11
4.4.	Funciones con objetos como argumentos	12
4.4.1.	El problema: tipado en línea (No recomendado para objetos complejos).....	12
4.4.2.	La solución: uso de interfaces como contratos de argumento	12
5.	Desestructuración (objetos, arreglos y argumentos).....	13
5.1.	Desestructuración de objetos.....	13
5.1.1.	El problema: código repetitivo.....	13
5.1.2.	La solución: extracción directa.....	13
5.1.3.	Peligro común: renombrar variables vs tipar	13
5.2.	Desestructuración de arreglos	14
5.2.1.	¿Cómo saltar elementos en un arreglo?	14
5.2.2.	Diferencia clave entre desestructurar objetos y arreglos	14
5.3.	Desestructuración de argumentos en funciones.....	14
5.3.1.	El problema: código saturado en la función.....	14
5.3.2.	La solución: desestructurar directamente en los parámetros	15
6.	Importaciones y exportaciones (modularización)	15
6.1.	Exportaciones nombradas (named exports)	15
6.1.1.	¿Por qué son la opción preferida de TypeScript?.....	16
6.2.	Exportaciones por defecto (default exports)	16
6.2.1.	Los inconvenientes de las exportaciones por defecto	16
6.3.	La sintaxis de importación.....	17
6.3.1.	Importar elementos nombrados	17
6.3.2.	Importar elementos por defecto	17
6.3.3.	C. Renombrar elementos durante la importación (as).....	17
6.4.	Archivos de agrupación o archivos de barril (barrel files)	17
7.	Clases y programación orientada a objetos (POO).....	18
7.1.	Clases básicas e inicialización de propiedades	18
7.1.1.	El enfoque tradicional	18
7.1.2.	El enfoque moderno de TypeScript: propiedades de parámetro (shorthand).....	19
7.2.	Modificadores de acceso.....	19
7.3.	Extender una clase (herencia)	20
7.4.	Priorizar composición sobre herencia	21
7.4.1.	¿Por qué es mejor este enfoque?	21

8.	Tipos genéricos.....	22
8.1.	El problema: perder el tipo de dato con any	22
8.2.	La solución: sintaxis de un genérico (<T>)	22
8.3.	Interfaces genéricas	23
8.4.	Restricciones en los genéricos (extends)	24
9.	Decoradores	24
9.1.	¿Cómo funcionan los decoradores por dentro?	24
9.2.	Creación de un decorador de clase básico	25
9.3.	Decoradores que modifican el comportamiento (bloqueo de cambios).....	25
9.4.	8.4 Fábricas de decoradores (decorator factories)	26
10.	Encadenamiento opcional (optional chaining).....	26
10.1.	El problema: errores en tiempo de ejecución.....	27
10.2.	La solución: el operador ?	27
10.3.	Combinación con el operador de asignación por defecto (o ??)	28
10.4.	Aplicación en funciones y métodos	28

1. Preambulo

Antes de pasar al proyecto de Angular, necesitaremos entender los fundamentos de TypeScript, que comparte muchas características con JavaScript. En esta sección se reforzarán los conceptos esenciales de JavaScript moderno que utilizaremos constantemente en Angular y se introducirá TypeScript desde cero.

2. Tipos básicos y conceptos generales

2.1. ¿Qué es TypeScript y por qué lo utilizamos?

TypeScript es un **superset** (o superconjunto) de JavaScript. Esto significa que todo el código escrito en JavaScript es perfectamente válido en TypeScript, pero TypeScript añade características adicionales, siendo la principal el sistema de tipado estático.

Los navegadores web no pueden ejecutar TypeScript directamente, solo entienden JavaScript. Por lo tanto, TypeScript requiere un proceso de compilación (o transpilación) en el cual el compilador de TypeScript traduce los archivos .ts a código JavaScript estándar (.js) compatible con cualquier navegador.

La ventaja fundamental de este flujo es la prevención de errores:

- En JavaScript (tipado dinámico): Los errores de tipo (como intentar ejecutar un texto como si fuera una función) se descubren cuando el usuario final interactúa con la aplicación y esta falla en su navegador.
- En TypeScript (tipado estático): Los errores se detectan en tiempo de desarrollo. El propio editor de código o el compilador detendrán el proceso si detectan incoherencias en la asignación de datos.

2.2. Declaración de variables: tipado explícito vs. tipado inferido

2.2.1. Tipado explícito (anotación de tipo)

Ocurre cuando el programador define manualmente el tipo de dato utilizando la sintaxis de dos puntos (:) tras el nombre de la variable. Es la mejor práctica cuando se declaran variables sin un valor inicial inmediato.

```
let puntosDeVida: number;  
puntosDeVida = 100; // Correcto
```

2.2.2. Tipado inferido

Si declaramos una variable y le asignamos un valor inicial en la misma línea, TypeScript deduce automáticamente el tipo de dato y restringe la variable a ese tipo para siempre, sin necesidad de escribir la anotación explícita.

```
let nombreUsuario = 'Saitama'; // Typescript infiere que es de tipo 'string'  
  
// El editor mostrará un error de inmediato si intentamos hacer esto:  
nombreUsuario = 123; // Error: Tipo 'number' no es assignable a tipo 'string'
```

2.3. Los tipos primitivos en detalle

TypeScript utiliza los mismos tipos primitivos que JavaScript, pero bajo un control riguroso.

2.3.1. string (cadenas de texto)

Se utiliza para almacenar texto. Admite comillas simples ('), comillas dobles (") y plantillas literales (backticks `) para concatenar variables.

```
const heroe: string = 'Ryu';  
const mensaje: string = `El héroe seleccionado es ${heroe}`;
```

2.3.2. number (números)

A diferencia de otros lenguajes, no hay distinción entre números enteros (int) y decimales (float). Todos son tratados como números de punto flotante de doble precisión bajo el tipo number. También admite representaciones hexadecimales, binarias y octales.

```
let entero: number = 42;  
let decimal: number = 19.99;
```

2.3.3. boolean (booleanos)

Representa únicamente dos estados lógicos: verdadero (true) o falso (false). No admite valores falsos implícitos de JavaScript (como 0 o textos vacíos) cuando se requiere un valor estrictamente booleano.

```
let estaActivo: boolean = true;
```

2.4. El tipo any: ¿qué es y por qué se debe evitar?

El tipo any es un tipo de datos comodín. Al declarar una variable como any, le indicamos al compilador de TypeScript que desactive por completo la comprobación de tipos para esa variable.

2.4.1. ¿Por qué existe?

Fue diseñado para facilitar la migración de proyectos antiguos de JavaScript a TypeScript de forma progresiva, y para interactuar con bibliotecas externas que no disponen de definiciones de tipo.

2.4.2. ¿Por qué se considera una mala práctica en código nuevo?

1. Anula el propósito de TypeScript: Si se utiliza `any`, el código vuelve a comportarse como JavaScript dinámico, perdiendo toda la seguridad frente a errores de escritura.
2. Propaga la inseguridad (efecto contagio): Si se asigna una variable de tipo `any` a otra variable que tiene un tipo definido, esta puede introducir datos incorrectos sin que el compilador emita advertencias.
3. Pérdida de asistencia en el editor: Al usar `any`, el editor de código pierde la capacidad de sugerir métodos autocompletados o autocompletar propiedades, ya que no sabe qué estructura tiene la variable.

```
// Ejemplo del peligro de 'any'
let datosApi: any = { nombre: 'Goku' };

// El compilador no mostrará ningún error al escribir esta línea,
// pero el programa fallará en el navegador porque el método "volar" no existe:
datosApi.volar();
```

2.4.3. La alternativa segura: unknown

Si realmente no sabemos qué tipo de datos va a recibir de una fuente externa (como una consulta a una base de datos), se debe utilizar el tipo `unknown`. A diferencia de `any`, TypeScript no nos permitirá interactuar con una variable `unknown` hasta que comprobemos explícitamente qué tipo de dato contiene mediante una validación.

```
let valorRecibido: unknown = 'Hola Mundo';

// Esto dará un error de compilación inmediato:
// console.log(valorRecibido.length);

// Para poder usarlo, debemos verificar el tipo primero:
if (typeof valorRecibido === 'string') {
  console.log(valorRecibido.length); // Ahora es seguro y está permitido
}
```

3. Objetos, arreglos e interfaces

En JavaScript, los objetos y los arreglos son estructuras dinámicas: podemos agregar, quitar o modificar sus propiedades y elementos en cualquier momento de la ejecución. En esta sección veremos cómo TypeScript introduce orden y previsibilidad a estas estructuras sin quitarles su flexibilidad.

3.1. Arreglos (arrays) en profundidad

Un arreglo es una colección ordenada de elementos. En JavaScript, un mismo arreglo puede contener simultáneamente textos, números, booleanos y objetos. En TypeScript, para mantener la seguridad de nuestro código, debemos restringir qué tipo de datos permitimos dentro de una colección.

3.1.1. Sintaxis básica de un arreglo

Para definir un arreglo que solo acepte un tipo de dato específico, escribimos el tipo seguido de corchetes ([]).

```
const habilidades: string[] = ['Volar', 'Esquivar', 'Curar'];

// Si intentamos añadir un número, TypeScript nos avisará del error:
// habilidades.push(100); // Error: El argumento de tipo 'number' no es asignable
// al parámetro de tipo 'string'.
```

3.1.2. Arreglos con tipos mixtos (unión de tipos)

Si realmente necesitamos que un arreglo almacene más de un tipo de dato, podemos utilizar la **unión de tipos** envolviendo los tipos permitidos entre paréntesis antes de los corchetes.

```
// Este arreglo permite almacenar tanto textos como números
const datosMixtos: (string | number)[] = ['Goku', 9000, 'Vegeta', 8500];
```

Debemos tener cuidado con la sintaxis. Escribir `string | number[]` (sin paréntesis) significa que la variable puede ser un solo texto o un arreglo de números. Colocar los paréntesis `(string | number[])` es lo que indica que es un arreglo donde cada posición puede ser un texto o un número.

3.1.3. Arreglos inmutables (readonly)

A veces queremos asegurarnos de que los elementos de un arreglo no se modifiquen, agreguen ni eliminen después de su creación (por ejemplo, una lista de configuraciones fijas). Para lograr esto, usaremos el modificador `readonly`.

```
const nombresFijos: readonly string[] = ['Admin', 'Soporte'];

// Cualquier intento de alterar el arreglo fallará en tiempo de desarrollo:
// nombresFijos.push('Usuario'); // Error: La propiedad 'push' no existe en el
// tipo 'readonly string[]'.
```

3.2. Objetos literales e inferencia de estructura

En JavaScript, podemos crear un objeto y añadirle propiedades sobre la marcha de esta forma:

```
// JavaScript tradicional
const heroe = { nombre: 'Goku' };
heroe.edad = 35; // Totalmente válido en JS
```

Sin embargo, en TypeScript, cuando creamos un objeto literal, el compilador **infiere su estructura exacta** y la sella de forma virtual.


```
// TypeScript
const heroe = {
  nombre: 'Goku',
  puntosVida: 100
};

// Si intentamos añadir una propiedad que no se definió al inicio:
// heroe.edad = 35; // Error: La propiedad 'edad' no existe en el tipo '{ nombre:
// string; puntosVida: number; }'.
```

3.2.1. ¿Por qué TypeScript bloquea la adición de propiedades?

TypeScript hace esto para protegernos de errores de escritura (typos) y asegurar que el objeto mantenga una estructura predecible. Si intentamos escribir `heroe.puntosVda = 80` (escribiendo mal la propiedad), en JavaScript se crearía una propiedad nueva silenciosamente y el juego o aplicación fallaría porque el código buscaría `puntosVida`. TypeScript detecta que `puntosVda` no existe en la definición original y nos detiene antes de que cometamos el error.

3.3. Interfaces

Cuando empezamos a trabajar con objetos más complejos o queremos reutilizar la misma estructura en diferentes partes de nuestra aplicación, escribir los tipos en línea se vuelve caótico. Para solucionar esto, utilizamos las **Interfaces**.

Una interfaz es un "contrato" o un molde que define qué propiedades debe tener un objeto y de qué tipo debe ser cada una. Las interfaces solo existen durante la fase de desarrollo para que TypeScript valide nuestro código; una vez transpilado a JavaScript, las interfaces desaparecen por completo, lo que significa que no añaden peso extra a nuestra aplicación final.

3.3.1. Declaración y uso básico

```
interface Personaje {
  nombre: string;
  puntosVida: number;
  habilidades: string[];
}

// Creamos un objeto y le asignamos la interfaz como su tipo:
const heroe: Personaje = {
  nombre: 'Spiderman',
  puntosVida: 80,
  habilidades: ['Tregar', 'Sentido arácnido']
};
```

3.3.2. Propiedades opcionales (?)

No siempre conocemos o necesitamos todos los datos de un objeto desde el principio. Para declarar que una

propiedad no es obligatoria, añadimos un signo de interrogación (?) justo antes de los dos puntos en la interfaz.

```
interface Personaje {
  nombre: string;
  puntosVida: number;
  habilidades: string[];
  puebloNatal?: string; // Propiedad opcional
}

// Este objeto es válido aunque no definamos "puebloNatal":
const heroe2: Personaje = {
  nombre: 'Iron Man',
  puntosVida: 95,
  habilidades: ['Volar', 'Disparar']
};
```

3.3.3. Propiedades de solo lectura (readonly en interfaces)

Al igual que con los arreglos, podemos proteger propiedades específicas de un objeto para que no puedan ser modificadas después de que el objeto haya sido creado (por ejemplo, un identificador único de base de datos).

```
interface Usuario {
  readonly id: string; // No se puede cambiar una vez asignado
  nombre: string;
}

const usuario: Usuario = {
  id: 'USR-12345',
  nombre: 'Ana'
};

usuario.nombre = 'Ana María'; // Permitido
// usuario.id = 'USR-99999'; // Error: No se puede asignar a 'id' porque es una
// propiedad de solo lectura.
```

4. Funciones y sus argumentos

En JavaScript, las funciones son muy flexibles: pueden recibir cualquier número de argumentos (sin importar si los definimos en la declaración o no) y pueden retornar cualquier tipo de dato de forma dinámica. En TypeScript, esta flexibilidad puede convertirse en una fuente de errores difíciles de rastrear. Por ello, aplicamos un control riguroso tanto a lo que entra en una función (parámetros) como a lo que sale de ella (retorno).

4.1. Funciones básicas: tipado de entradas y salidas

Cuando definimos una función en TypeScript, debemos indicar de forma explícita qué tipo de datos esperamos recibir y qué tipo de datos va a devolver la función.

```
// Declaración de función tradicional tipada
function sumar(a: number, b: number): number {
  return a + b;
}
```

En este ejemplo:

- **a: number** y **b: number** aseguran que la función solo acepte números como argumentos. Si intentamos pasar una cadena de texto, el compilador nos detendrá.
- El **: number** que se encuentra justo antes de la llave de apertura **{** define el **tipo de retorno**. Esto garantiza que la función devuelva obligatoriamente un número.

4.1.1. ¿Por qué es importante tipar el retorno?

Si no tipamos el retorno, TypeScript intentará inferirlo automáticamente. Sin embargo, tiparlo de forma explícita es una excelente práctica por dos motivos:

1. **Evitamos errores internos:** Si modificamos la lógica de la función y accidentalmente devolvemos un texto en lugar de un número, TypeScript nos mostrará un error de inmediato dentro de la propia función.
2. **Documentación del código:** Cualquiera que use nuestra función sabrá exactamente qué esperar de ella sin necesidad de leer todo el código interno.

4.1.2. El tipo de retorno void

Si una función realiza una acción (como imprimir en consola o guardar un dato) pero no devuelve ningún valor mediante la palabra clave **return**, su tipo de retorno debe ser **void** (vacío).

```
function mostrarMensaje(mensaje: string): void {
  console.log(mensaje);
}
```

En JavaScript, una función que no tiene un **return** explícito devuelve **undefined** de forma predeterminada durante la ejecución. En TypeScript, usamos **void** para indicar que el desarrollador no tiene intención de usar el valor de retorno de esa función, lo que ofrece un nivel de seguridad semántica mayor que usar simplemente **undefined**.

4.2. Funciones de flecha (arrow functions)

Las funciones de flecha se utilizan constantemente en el desarrollo moderno. La sintaxis para tiparlas sigue la misma lógica, pero la colocación de los tipos varía ligeramente.

```
// Tipado en una función de flecha
const multiplicar = (a: number, b: number): number => {
  return a * b;
};
```

Si la función de flecha se escribe en una sola línea (retorno implícito), mantenemos el tipado de los parámetros en los paréntesis:

```
const duplicar = (valor: number): number => valor * 2;
```

4.3. Argumentos opcionales y por defecto

En JavaScript, si llamamos a una función sin pasarle todos los parámetros que solicita, estos parámetros toman el valor de undefined y la ejecución continúa, lo que suele provocar errores matemáticos (como obtener un resultado NaN). En TypeScript, todos los parámetros declarados son **obligatorios** de forma predeterminada.

Para flexibilizar este comportamiento, disponemos de dos herramientas:

4.3.1. Argumentos opcionales (?)

Podemos indicar que un parámetro no es obligatorio añadiendo un signo de interrogación (?) tras su nombre. En estos casos, TypeScript nos obliga a verificar si el argumento existe antes de usarlo, previniendo fallos en la ejecución.

```
function saludar(nombre: string, apellido?: string): string {
  if (apellido) {
    return `Hola, ${nombre} ${apellido}`;
  }
  return `Hola, ${nombre}`;
}
```

4.3.2. Argumentos por defecto (valores predeterminados)

Si queremos que un parámetro sea opcional pero tenga un valor asignado automáticamente en caso de que no lo enviemos, le asignamos un valor inicial con el operador =.

```
function configurarConexion(servidor: string, puerto: number = 8080): string {
  return `Conectando a ${servidor}:${puerto}`;
}

configurarConexion('localhost'); // Resultado: "Conectando a localhost:8080"
configurarConexion('localhost', 3000); // Resultado: "Conectando a localhost:3000"
```

Regla de orden estricta: En las funciones de TypeScript, los parámetros opcionales y los parámetros con valores por defecto deben colocarse siempre al final de la lista de argumentos. No podemos definir un parámetro opcional antes de uno obligatorio, ya que el compilador no sabría cómo emparejar los valores cuando

llamemos a la función.

4.4. Funciones con objetos como argumentos

Cuando una función necesita recibir un objeto con múltiples propiedades, tiparlo directamente dentro de los parámetros de la función puede hacer que el código sea muy difícil de leer y mantener.

4.4.1. El problema: tipado en línea (No recomendado para objetos complejos)

```
function registrarUsuario(usuario: { id: string; correo: string; activo: boolean }): void {  
  console.log(`Registrando a: ${usuario.correo}`);  
}
```

Esta sintaxis satura la firma de la función y hace que el código sea confuso. Además, si necesitamos crear otra función que procese el mismo tipo de usuario, tendríamos que duplicar todo ese bloque de tipos.

4.4.2. La solución: uso de interfaces como contratos de argumento

Para mantener el código limpio, legible y reutilizable, creamos una interfaz que describa la estructura del objeto y luego la aplicamos como el tipo del parámetro de nuestra función.

```
// 1. Definimos la estructura del objeto mediante una interfaz  
interface Usuario {  
  id: string;  
  correo: string;  
  activo: boolean;  
}  
  
// 2. Usamos la interfaz para tipar el argumento de la función  
function registrarUsuario(usuario: Usuario): void {  
  console.log(`Registrando a: ${usuario.correo}`);  
}  
  
// 3. Creamos el objeto respetando la interfaz  
const nuevoUsuario: Usuario = {  
  id: 'USR-99',  
  correo: 'correo@ejemplo.com',  
  activo: true  
};  
  
// 4. Invocamos la función de forma segura  
registrarUsuario(nuevoUsuario);
```

Si intentamos enviar un objeto que no cumpla con las propiedades exigidas por la interfaz **Usuario** (por ejemplo, si nos falta la propiedad **activo**), TypeScript impedirá la compilación y nos señalará exactamente qué propiedad falta o está mal escrita.

5. Desestructuración (objetos, arreglos y argumentos)

La desestructuración es una característica de JavaScript moderno que nos permite extraer valores de objetos o elementos de arreglos y asignarlos a variables individuales de una forma mucho más limpia y directa.

En TypeScript, esta práctica no solo hace que el código sea más legible, sino que se beneficia por completo del sistema de tipado, ya que las variables que extraemos heredan automáticamente el tipo de dato que tenían en su estructura original.

5.1. Desestructuración de objetos

Cuando trabajamos con objetos, a menudo necesitamos acceder a sus propiedades varias veces. Hacer esto repitiendo el nombre del objeto una y otra vez puede saturar el código.

5.1.1. El problema: código repetitivo

```
interface Reproductor {  
  volumen: number;  
  segundo: number;  
  cancion: string;  
}  
  
const reproductor: Reproductor = {  
  volumen: 90,  
  segundo: 36,  
  cancion: 'Clair de Lune'  
};  
  
// Sin desestructuración, tenemos que repetir "reproductor" constantemente:  
console.log('Canción actual: ', reproductor.cancion);  
console.log('Volumen: ', reproductor.volumen);
```

5.1.2. La solución: extracción directa

Con la desestructuración, podemos crear variables locales con el mismo nombre de las propiedades del objeto en una sola línea de código:

```
const { cancion, volumen } = reproductor;  
  
// Ahora usamos las variables directamente:  
console.log('Canción actual: ', cancion);  
console.log('Volumen: ', volumen);
```

5.1.3. Peligro común: renombrar variables vs tipar

Un error muy frecuente cuando empezamos con TypeScript es intentar definir el tipo de dato dentro de la llave de desestructuración, pensando que funciona como una declaración normal:

```
// ¡ESTO ES UN ERROR!  
const { cancion: string } = reproductor;
```

5.2. Desestructuración de arreglos

A diferencia de los objetos, donde extraemos los valores buscando el nombre de su propiedad, en los arreglos realizamos la extracción basándonos estrictamente en la **posición física (el índice)** de los elementos.

```
const personajes: string[] = ['Goku', 'Vegeta', 'Trunks'];  
  
// Extraemos los elementos por orden de posición  
const [goku, vegeta, trunks] = personajes;  
  
console.log(vegeta); // Imprime: 'Vegeta'
```

5.2.1. ¿Cómo saltar elementos en un arreglo?

No siempre vamos a querer extraer todos los elementos de un arreglo. Si solo nos interesa el tercer personaje, no estamos obligados a crear variables para los dos primeros. Podemos dejar esos espacios vacíos separados por comas:

```
// Dejamos los primeros dos espacios vacíos  
const [ , , tercerPersonaje] = personajes;  
  
console.log(tercerPersonaje); // Imprime: 'Trunks'
```

5.2.2. Diferencia clave entre desestructurar objetos y arreglos

- **En objetos:** El orden no importa, lo que importa es el nombre exacto de la propiedad.
- **En arreglos:** El nombre que le demos a la variable no importa (podemos llamarla como queramos), lo que importa es la posición que ocupa dentro de los corchetes.

5.3. Desestructuración de argumentos en funciones

Esta es una de las técnicas más utilizadas cuando una función recibe un objeto como parámetro, pero solo necesitamos usar una o dos de sus propiedades dentro del cuerpo de la función.

5.3.1. El problema: código saturado en la función

```
interface Producto {
  descripcion: string;
  precio: number;
}

const telefono: Producto = {
  descripcion: 'Teléfono inteligente',
  precio: 350
};

// La función recibe el objeto completo, pero solo usa el precio
function calcularImpuesto(producto: Producto): number {
  return producto.precio * 0.15;
}
```

5.3.2. La solución: desestructurar directamente en los parámetros

Podemos aplicar las llaves de desestructuración directamente en la firma de la función. De esta manera, extraemos solo lo que nos interesa y seguimos manteniendo el tipado del objeto de forma segura:

```
// Colocamos las llaves dentro del parámetro para extraer "precio" y "descripcion"
function calcularImpuesto({ precio, descripcion }: Producto): number {
  console.log(`Calculando el impuesto para: ${descripcion}`);
  return precio * 0.15;
}

// Para llamar a la función, seguimos enviando el objeto completo
calcularImpuesto(telefono);
```

Esta sintaxis es muy potente porque nos evita tener que escribir `producto.precio` o declarar variables adicionales dentro de la función, manteniendo el código limpio y completamente tipado.

6. Importaciones y exportaciones (modularización)

Cuando nuestras aplicaciones empiezan a crecer, es imposible y poco práctico mantener todo nuestro código dentro de un único archivo. La solución es dividir nuestra aplicación en múltiples archivos especializados. En el desarrollo de software, a cada uno de estos archivos individuales lo conocemos como un **módulo**.

Por defecto, en TypeScript (y en JavaScript moderno), cada archivo es un compartimento aislado. Las variables, funciones, clases o interfaces que creamos en un archivo no son visibles para los demás archivos a menos que las **exportemos** explícitamente. De igual forma, para usar algo creado en otro archivo, debemos **importarlo**.

6.1. Exportaciones nombradas (named exports)

Las exportaciones nombradas nos permiten exportar múltiples elementos (funciones, interfaces, variables, clases) de un mismo archivo. Para utilizarlas, simplemente anteponeamos la palabra clave **export** antes de la declaración de lo que queremos compartir.

Ejemplo de archivo de origen (**src/helpers/calculos.ts**):


```
export interface ResultadoFactura {
  subtotal: number;
  impuesto: number;
  total: number;
}

export const ISV: number = 0.15;

export function calcularTotal(subtotal: number): ResultadoFactura {
  const impuesto = subtotal * ISV;
  return {
    subtotal,
    impuesto,
    total: subtotal + impuesto
  };
}
```

6.1.1. ¿Por qué son la opción preferida de TypeScript?

- **Consistencia de nombres:** Al importarlos en otro archivo, estamos obligados a utilizar el mismo nombre exacto con el que fueron exportados. Esto evita confusiones y mantiene el código unificado.
- **Facilidad de refactorización:** Si el día de mañana cambiamos el nombre de la función calcularTotal a obtenerTotalFactura, nuestro editor de código podrá buscar y renombrar automáticamente todas las importaciones en nuestro proyecto sin romper nada.
- **Asistencia en el autocompletado:** Al empezar a escribir las llaves de importación, el editor nos sugerirá exactamente qué funciones e interfaces están disponibles en ese archivo.

6.2. Exportaciones por defecto (default exports)

Una exportación por defecto nos permite definir un único elemento principal que será lo representativo de ese archivo. Para ello, utilizamos la combinación de palabras **export default**.

Ejemplo de archivo de origen (**src/services/authentication.ts**):

```
class ServicioAutenticacion {
  login(usuario: string) {
    return `Usuario ${usuario} autenticado`;
  }
}

// Exportación única por defecto
export default ServicioAutenticacion;
```

6.2.1. Los inconvenientes de las exportaciones por defecto

Aunque son muy comunes en JavaScript, en TypeScript presentan ciertas desventajas que debemos considerar:

- **Nombres arbitrarios al importar:** Cuando importamos un elemento por defecto, podemos ponerle el nombre que queramos en el archivo de destino. Esto puede provocar que un mismo servicio se llame de formas distintas en diferentes partes de nuestra aplicación, dificultando el mantenimiento del código.

- **Peor soporte para refactorización:** Si renombramos la clase original, el editor no siempre actualizará automáticamente los nombres libres que les hayamos dado en las importaciones.

6.3. La sintaxis de importación

Para consumir los elementos que hemos exportado en otros archivos, utilizamos la palabra clave **import** al inicio de nuestro archivo de destino.

6.3.1. Importar elementos nombrados

Para importar elementos específicos, los colocamos dentro de llaves **{ }** y especificamos la ruta relativa de donde se encuentra el archivo (sin la extensión **.ts**).

```
// Importamos solo lo que necesitamos de nuestro archivo de cálculos
import { calcularTotal, ResultadoFactura } from './helpers/calculos';

const miFactura: ResultadoFactura = calcularTotal(100);
```

6.3.2. Importar elementos por defecto

Para importar un elemento que fue exportado por defecto, no utilizamos las llaves. Le asignamos su nombre directamente.

```
// Importamos la clase asignándole un nombre (no lleva llaves)
import Autenticador from './services/autenticacion';

const auth = new Autenticador();
```

6.3.3. C. Renombrar elementos durante la importacion (as)

Si importamos un elemento y su nombre entra en conflicto con otra variable o función que ya tenemos en nuestro archivo, podemos renombrarlo usando la palabra clave **as**.

```
import { calcularTotal as obtenerSumaFactura } from './helpers/calculos';

const total = obtenerSumaFactura(100);
```

6.4. Archivos de agrupación o archivos de barril (barrel files)

Cuando nuestro proyecto crece, podemos terminar con una enorme lista de importaciones al principio de nuestros archivos, lo cual satura visualmente nuestro código:

```
// Código saturado de importaciones individuales
import { ClaseA } from './moduloA';
import { ClaseB } from './moduloB';
import { ClaseC } from './moduloC';
```

Ejemplo de un archivo de barril (**src/services/index.ts**):

```
export { ServicioA } from './servicioA';
export { ServicioB } from './servicioB';
export { ServicioC } from './servicioC';
```

Ahora, cuando necesitemos consumir cualquiera de estos servicios desde cualquier otra parte de nuestra aplicación, solo necesitaremos una única línea de importación apuntando a la carpeta contenedora:

```
// El compilador busca automáticamente el archivo index.ts dentro de la carpeta
"services"
import { ServicioA, ServicioB, ServicioC } from './services';
```

7. Clases y programación orientada a objetos (POO)

Cuando trabajamos con clases en JavaScript, solo estamos usando una forma más limpia de organizar el código, pero por dentro el lenguaje sigue funcionando exactamente igual que siempre. El verdadero inconveniente es que JavaScript no nos deja proteger los datos de un objeto: cualquier persona puede leer o cambiar una propiedad desde fuera de la clase sin ninguna restricción, lo que expone nuestro código a errores.

TypeScript transforma las clases en herramientas robustas para la programación orientada a objetos, permitiéndonos definir con precisión cómo se estructuran nuestros objetos, quién puede acceder a sus datos y cómo se relacionan entre sí.

7.1. Clases básicas e inicialización de propiedades

En otros lenguajes de programación, y en el JavaScript tradicional, definir una clase requiere declarar las propiedades, recibirlas en el constructor y luego asignarlas una a una usando la palabra clave **this**.

7.1.1. El enfoque tradicional

```
class Heroe Tradicional {
  // 1. Declaramos las propiedades y sus tipos
  public nombre: string;
  public poder: string;

  // 2. Las recibimos en el constructor
  constructor(nombre: string, poder: string) {
    // 3. Las asignamos manualmente
    this.nombre = nombre;
    this.poder = poder;
  }
}
```

7.1.2. El enfoque moderno de TypeScript: propiedades de parámetro (shorthand)

TypeScript introduce un atajo sintáctico sumamente potente. Si añadimos un modificador de acceso (como **public** o **private**) directamente antes de los parámetros del constructor, TypeScript hace tres cosas automáticamente por nosotros:

1. Declara la propiedad en la clase.
2. Recibe el valor en el constructor.
3. Asigna el valor a **this.nombre** de forma implícita.

```
class Heroe {  
  // Este constructor hace exactamente lo mismo que el ejemplo largo de arriba  
  constructor(  
    public nombre: string,  
    public poder: string,  
    public edad?: number // Propiedad opcional  
  ) {}  
}  
  
const ironman = new Heroe('Iron Man', 'Tecnología', 45);
```

7.2. Modificadores de acceso

Los modificadores de acceso nos permiten controlar la visibilidad de las propiedades y métodos de una clase. Esto es fundamental para aplicar el concepto de **encapsulación**, asegurando que partes sensibles de nuestro código no sean modificadas de forma errónea desde el exterior.

Tenemos tres modificadores principales en TypeScript:

- **public**: Visibles en cualquier lugar. Es el comportamiento por defecto. La propiedad o método se puede leer y modificar desde fuera de la clase.
- **private**: Visible solo dentro de la clase. La propiedad o método solo existe y se puede usar dentro de las llaves `{}` de la clase donde fue declarada.
- **protected**: Visible dentro de la clase y sus herederos. Es idéntica a **private**, pero también permite que las clases que hereden de ella (clases hijas) puedan acceder a la propiedad.

Ejemplo de uso de modificaciones:

```
class CuentaBancaria {
  constructor(
    public titular: string,      // Accesible desde fuera
    private saldo: number,      // Protegido contra modificaciones externas directas
    protected identificador: string // Accesible aquí y en clases que hereden de esta
  ) {}

  // Método público para interactuar de forma segura con el saldo privado
  public depositar(cantidad: number): void {
    if (cantidad > 0) {
      this.saldo += cantidad;
    }
  }

  public obtenerSaldo(): string {
    return `El saldo actual es de ${this.saldo}`;
  }
}

const miCuenta = new CuentaBancaria('Sofía', 1000, 'ID-992');
miCuenta.depositar(500); // Correcto

// Intentar modificar el saldo directamente dará un error de compilación:
// miCuenta.saldo = 50000; // Error: La propiedad 'saldo' es privada.
```

7.3. Extender una clase (herencia)

La herencia nos permite crear una nueva clase (clase hija) basada en una clase existente (clase padre), heredando todas sus propiedades y métodos públicos o protegidos. Para ello utilizamos la palabra clave **extends**.

Cuando la clase hija tiene su propio constructor, estamos obligados a llamar al constructor de la clase padre utilizando la función **super()** antes de intentar usar **this**.

```
class Persona {
  constructor(
    public nombre: string,
    public direccion: string
  ) {}
}

class Empleado extends Persona {
  constructor(
    public sueldo: number,
    // Recibimos las propiedades que necesita la clase padre
    nombre: string,
    direccion: string
  ) {
    // super() envía los parámetros requeridos al constructor de Persona
    super(nombre, direccion);
  }
}

const nuevoEmpleado = new Empleado(2500, 'Carlos', 'Calle Mayor 12');
```

7.4. Priorizar composición sobre herencia

Un error muy común en el diseño de software es abusar de la herencia para reutilizar código. Si creamos árboles de herencia demasiado profundos (por ejemplo: **ClaseA** hereda de **ClaseB**, que hereda de **ClaseC** y así sucesivamente), nuestro código se vuelve extremadamente rígido. Cualquier cambio en la clase padre puede romper de forma inesperada todas las clases hijas.

Para evitar esto, la arquitectura de software moderna recomienda **priorizar la composición sobre la herencia**.

- **Herencia** (Relación "Es un"): Un Heroe *es una* Persona.
- **Composición** (Relación "Tiene un"): Un Automovil *tiene un* Motor (no "es un" motor).

En lugar de heredar el comportamiento de otra clase, pasamos una instancia de esa clase como una propiedad para construir un sistema mucho más flexible y fácil de cambiar en el futuro.

Ejemplo de composición:

```
// 1. Creamos clases independientes especializadas
class MotorElectrico {
  public encender(): string {
    return 'Motor eléctrico iniciado en silencio...';
  }
}

class GPS {
  public obtenerCoordenadas(): string {
    return 'Lat: 40.41, Lon: -3.70';
  }
}

// 2. En lugar de heredar de Motor o GPS, componemos el Automóvil usándolos
class Automovil {
  // El automóvil TIENE un motor y TIENE un GPS como propiedades
  constructor(
    public marca: string,
    public motor: MotorElectrico,
    public sistemaNavegacion: GPS
  ) {}

  public arrancarYViajar(): void {
    console.log(this.motor.encender());
    console.log(`Ruta fijada en: ${this.sistemaNavegacion.obtenerCoordenadas()}`);
  }
}

// 3. Instanciamos las piezas y armamos nuestro objeto
const motor = new MotorElectrico();
const gps = new GPS();

const miAuto = new Automovil('Tesla', motor, gps);
miAuto.arrancarYViajar();
```

7.4.1. ¿Por qué es mejor este enfoque?

Si el día de mañana queremos que nuestro **Automovil** utilice un **MotorCombustion** en lugar de un **MotorE-**

lectrico, no tenemos que reescribir la estructura heredada de la clase. Al estar compuesto de piezas independientes, solo necesitamos cambiar la pieza que le pasamos al constructor sin alterar el funcionamiento interno del automóvil.

8. Tipos genéricos

En el desarrollo de software, a menudo nos encontramos con la necesidad de escribir código que sea reutilizable, es decir, que pueda funcionar con diferentes tipos de datos.

Si queremos crear una función que simplemente reciba un argumento y lo devuelva, podríamos vernos tentados a usar el tipo `any` para que acepte cualquier cosa. Sin embargo, como ya hemos aprendido, usar `any` hace que perdamos toda la seguridad y el autocompletado de TypeScript.

Para solucionar este dilema de forma elegante, TypeScript introduce los **genéricos**. Los genéricos nos permiten crear funciones, clases o interfaces que son completamente flexibles respecto al tipo de dato con el que trabajan, pero que **mantienen la seguridad del tipo original** una vez que se ejecutan.

8.1. El problema: perder el tipo de dato con `any`

Imaginemos que queremos crear una función que tome un argumento y nos lo devuelva exactamente como entró.

```
function retornarElemento(argumento: any): any {
  return argumento;
}

// Probamos la función con diferentes tipos de datos
const unTexto = retornarElemento('Hola Mundo');
const unNumero = retornarElemento(150);
```

Como la función devuelve un tipo `any`, TypeScript ya no sabe que contiene la variable `unTexto`. Si intentamos escribir `unTexto`. en nuestro editor, no nos sugerirá métodos propios de las cadenas de texto (como `.length` o `.toUpperCase()`), porque para el compilador esa variable es un misterio. Peor aún, el compilador no nos detendrá si intentamos tratar `unNumero` como si fuera un texto, lo que causará un error en nuestro programa cuando se ejecute.

8.2. La solución: sintaxis de un genérico (`<T>`)

Un genérico actúa como una “variable de tipo”. En lugar de fijar un tipo como `string` o usar `any`, definimos una letra (por convención se usa la `T` de Type) encerrada entre llaves angulares `<T>` justo antes de los parámetros de la función.

```
// Le decimos a la función que va a manejar un tipo genérico "T"
function retornarElemento<T>(argumento: T): T {
  return argumento;
}
```

Cuando llamamos a la función y le pasamos un valor, TypeScript “captura” automáticamente el tipo de ese

valor y reemplaza la **T** con ese tipo de forma interna y temporal. Si le pasamos un **string**, la función se comporta exactamente como si estuviera escrita de la forma **(argumento: string): string**.

```
// TypeScript detecta automáticamente que "T" es de tipo 'string'
const unTexto = retornarElemento('Hola Mundo');

// Ahora el editor sabe perfectamente que es un string y nos dará autocompletado:
console.log(unTexto.toUpperCase()); // Correcto y seguro

// Aquí, TypeScript detecta que "T" es de tipo 'number'
const unNumero = retornarElemento(150);

// Si intentamos hacer esto, el editor nos dará un error inmediatamente:
// console.log(unNumero.toUpperCase()); // Error: La propiedad 'toUpperCase' no
// existe en el tipo 'number'.
```

8.3. Interfaces genéricas

Los genéricos no se limitan a las funciones; también podemos aplicarlos a las interfaces. Esto es muy útil cuando tenemos una estructura de datos que comparte algunas propiedades comunes, pero una propiedad específica puede cambiar de tipo según el contexto.

Imaginemos que queremos representar la respuesta de una petición a una base de datos o un servidor de internet:

```
// Definimos una interfaz con un tipo genérico "T"
interface RespuestaServidor<T> {
  codigoEstado: number;
  exitoso: boolean;
  datos: T; // Los datos pueden ser de cualquier tipo que decidamos al usar la
  interfaz
}

// 1. Uso para una respuesta que devuelve los datos de un Usuario
interface Usuario {
  nombre: string;
  correo: string;
}

const respuestaUsuario: RespuestaServidor<Usuario> = {
  codigoEstado: 200,
  exitoso: true,
  datos: {
    nombre: 'Carlos',
    correo: 'carlos@ejemplo.com'
  }
};

// 2. Uso para una respuesta que simplemente devuelve un arreglo de números
const respuestaPuntajes: RespuestaServidor<number[]> = {
  codigoEstado: 200,
  exitoso: true,
  datos: [100, 85, 90]
};
```


Gracias a esto, no tenemos que crear interfaces separadas como **RespuestaUsuario** y **RespuestaPuntajes**. Una sola interfaz genérica se adapta perfectamente a cualquier tipo de dato que necesitemos transportar en la propiedad **datos**, manteniendo nuestro código increíblemente limpio y tipado de extremo a extremo.

8.4. Restricciones en los genéricos (extends)

A veces, permitir que un genérico acepte cualquier tipo es demasiado libre. Podríamos querer que nuestra función genérica solo acepte tipos de datos que cumplan con una estructura mínima. Para lograr esto, podemos restringir el genérico utilizando la palabra clave **extends**.

```
interface ConNombre {
  nombre: string;
}

// Restringimos "T" para que solo acepte tipos que al menos tengan la propiedad
"nombre"
function imprimirNombre<T extends ConNombre>(objeto: T): void {
  console.log(`El nombre es: ${objeto.nombre}`);
}

const heroe = { nombre: 'Logan', poder: 'Regeneración' };
const carro = { marca: 'Toyota', modelo: 2024 };

imprimirNombre(heroe); // Correcto: "heroe" tiene la propiedad "nombre"

// Esto fallará en tiempo de desarrollo:
// imprimirNombre(carro); // Error: El tipo de 'carro' no cumple con la restricción
// de 'ConNombre'.
```

9. Decoradores

Los decoradores son, en esencia, funciones especiales que nos permiten modificar, expandir o alterar el comportamiento de una clase, un método o una propiedad. Para usarlos, simplemente colocamos el símbolo arroba (@) seguido del nombre de la función justo encima de la clase o elemento que queremos transformar. Si ya conocemos Angular, sabemos que se usan constantemente (como **@Component** o **@Injectable**). Sin embargo, es fundamental entender qué hacen por dentro en TypeScript puro antes de verlos integrados en un framework.

9.1. ¿Cómo funcionan los decoradores por dentro?

Un error muy común es pensar que un decorador se ejecuta cada vez que creamos una nueva instancia de una clase (usando **new**). Esto no es así. Un decorador se ejecuta una sola vez, justo cuando el JavaScript lee la definición de la clase por primera vez al arrancar la aplicación.

Lo que hace TypeScript es tomar nuestra clase y pasársela como argumento a la función del decorador. De este modo, la función puede revisar la clase, añadirle nuevas propiedades, modificar sus métodos o incluso

sustituir la clase por completo por otra versión mejorada.

Al ser todavía una característica en desarrollo dentro del estándar de JavaScript, para que funcionen en TypeScript puro necesitamos activar una casilla en nuestro archivo de configuración **tsconfig.json** asegurándonos de que la línea **"experimentalDecorators": true** esté desbloqueada.

9.2. Creación de un decorador de clase básico

Vamos a crear un decorador sencillo. Imaginemos que queremos que varias clases de nuestra aplicación registren un mensaje en la consola de forma automática en cuanto el sistema las detecte.

```
// 1. Un decorador es una función normal que recibe el constructor de la clase
function RegistrarClase(constructor: Function) {
  console.log(`La clase ${constructor.name} ha sido detectada y registrada.`);
}

// 2. Aplicamos el decorador usando el símbolo "@"
@RegistrarClase
class ServicioUsuarios {
  constructor() {
    console.log('Instancia de ServicioUsuarios creada.');
```

9.3. Decoradores que modifican el comportamiento (bloqueo de cambios)

El verdadero potencial de los decoradores aparece cuando queremos alterar el funcionamiento real de una clase. Supongamos que queremos proteger una clase para que nadie pueda añadirle nuevas propiedades ni modificar sus métodos desde fuera una vez creada.

Para lograr esto, podemos usar herramientas nativas de JavaScript dentro de nuestro decorador, como **Object.seal**:

```
// Este decorador "sella" la clase para evitar que sea modificada en el futuro
function SellarClase(constructor: Function) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
  console.log(`La clase ${constructor.name} ahora está protegida contra modificaciones.`);
}

@SellarClase
class ConfiguracionSistema {
  public urlApi: string = 'https://api.ejemplo.com';
}

const config = new ConfiguracionSistema();

// Si alguien intenta alterar la estructura de la clase desde fuera, el sistema fallará:
// (config as any).nuevaPropiedad = 'intento de hackeo'; // Error en tiempo de ejecución
```

9.4. 8.4 Fábricas de decoradores (decorator factories)

A veces necesitamos que un decorador sea flexible y reciba parámetros externos para decidir qué hacer. Para lograrlo, creamos una función que envuelve al decorador (una fábrica). Esta función recibe nuestros parámetros y devuelve la función real del decorador.

```
// Esta función recibe un texto personalizado
function ValidarAcceso(rolRequerido: string) {
  // Devolvemos el decorador real
  return function(constructor: Function) {
    console.log(`Protegiendo la clase ${constructor.name}. Solo accesible para el rol: ${rolRequerido}`);
    // Aquí podríamos añadir lógica para bloquear la clase si el usuario actual no coincide
  };
}

// Pasamos el argumento directamente al usar el decorador
@ValidarAcceso('Administrador')
class PanelControlFinanzas {
  public datosSensibles = [100, 500, 2000];
}
```

Gracias a los decoradores, podemos evitar repetir el mismo código de validación, registro o protección en veinte clases diferentes. Simplemente creamos la lógica una vez y la "enganchamos" encima de cualquier clase que la necesite.

10. Encadenamiento opcional (optional chaining)

El encadenamiento opcional es una herramienta que nos ayuda a evitar que la aplicación falle por completo cuando intentamos acceder a propiedades de un objeto que, por algún motivo, no existe o está vacío (**null** o

undefined).

En el desarrollo web, es muy común consultar información de servidores externos o bases de datos. Si en algún momento esa información llega incompleta y nuestro código intenta leer un dato que no está ahí, el programa se detendrá inmediatamente con un error crítico en la consola del navegador.

10.1. El problema: errores en tiempo de ejecución

Imaginemos que estamos gestionando los datos de los pasajeros de una aerolínea. Algunos pasajeros viajan con hijos y otros no, por lo que la propiedad **hijos** es opcional en nuestra interfaz.

```
interface Pasajero {  
  nombre: string;  
  hijos?: string[]; // Propiedad opcional  
}  
  
const pasajero1: Pasajero = {  
  nombre: 'Ana'  
  // No tiene la propiedad "hijos" declarada  
};
```

Si queremos saber cuántos hijos tiene un pasajero para calcular el precio de un billete, podríamos vernos tentados a escribir esto:

```
// ¡ESTO CAUSARÁ UN ERROR CRÍTICO!  
const totalHijos = pasajero1.hijos.length;
```

Como **pasajero1.hijos** no existe, su valor es **undefined**. Al intentar leer la propiedad **.length** de algo que es **undefined**, JavaScript rompe la ejecución del programa por completo.

10.2. La solución: el operador ?.

TypeScript (y el JavaScript moderno) solucionan esto introduciendo el operador de encadenamiento opcional, que se escribe con un signo de interrogación seguido de un punto (**?.**).

Este operador le dice al programa que intente leer la propiedad que viene a continuación. Si la propiedad anterior es null o undefined, detiene la lectura inmediatamente y devuelve un undefined, pero sin romper la aplicación.

```
const totalHijos = pasajero1.hijos?.length;  
  
console.log(totalHijos); // Imprime: undefined (pero la aplicación sigue funcionando)
```

10.3. Combinación con el operador de asignación por defecto (|| o ??)

Devolver **undefined** nos salva de un error crítico, pero en la vida real, si estamos contando hijos o calculando un total, probablemente necesitemos un número real (como un **0**) en lugar de un vacío para poder seguir operando.

Para resolver esto, combinamos el encadenamiento opcional con el operador de coalescencia nula (??) o el operador lógico OR (||). Esto nos permite definir un valor de respaldo inmediatamente.

```
// Si la parte izquierda da "undefined" o "null", el operador "??" asigna un 0 automáticamente
const cuantosHijos = pasajero1.hijos?.length ?? 0;

console.log(cuantosHijos); // Imprime: 0
```

10.4. Aplicación en funciones y métodos

El encadenamiento opcional no solo sirve para las propiedades de los objetos, sino que también podemos usarlo para asegurarnos de que una función o un método existe antes de intentar ejecutarlo.

Imaginemos que recibimos la configuración de una interfaz donde un botón puede o no tener una acción asignada:

```
interface Boton {
  etiqueta: string;
  accion?: () => void; // Una función opcional que no devuelve nada
}

const botonGuardar: Boton = {
  etiqueta: 'Guardar'
  // No tiene acción definida
};

// Al añadir "?. " antes de los paréntesis, la función solo se ejecuta si existe.
// En este caso, como no existe, no hace nada y no rompe el programa.
botonGuardar.accion?.();
```